

API DOCUMENTATION

The load problem is the request, not the response.

Every time the user sends one new message, the client sends the full chat history again to the chat API. The API then sends that same growing history to Gemini. The HTTP response is small (only the new assistant text, or null). This document is English. It does not propose summarising old messages.

1. What is wrong with the current API shape

- **Request grows every turn.** Body = systemInstruction + all previous turns + new userMessage. Turn 2 is small. Turn 20 is large. Turn 50 is much larger. Same API name every time.
- **Gemini sees the same growth.** The backend forwards that history to generateContent. More input tokens, more cost, more chance of hitting the 30s timeout.
- **Response does not grow.** Success returns only the new reply text (maxOutputTokens 512). Failure returns null. Response size is not the load issue.
- **Firestore already has the messages** (app writes them). Sending history again on the API is a second copy.

2. Current API

Field	Value
Method / path (proxy)	POST /v1/chat
Upstream	POST https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent
Timeout	30 seconds
temperature / maxOutputTokens	0.7 / 512
Modes	role-play and free-chat (different systemInstruction, same endpoint)

3. Current request - this is the heavy part (every message)

POST /v1/chat

history is the full past conversation. It is sent again on message 3, 4, 5, ... not only the first time.

```
{
  "systemInstruction": "You are a friendly language-learning conversation partner. Topic: Restaurant. Language: Spanish. ...",
  "userMessage": "Una mesa para dos",
  "history": [
    { "text": "Bienvenido. Cuantas personas?", "isUser": false },
    { "text": "Somos dos", "isUser": true },
    { "text": "Interior o terraza?", "isUser": false }
    /* every older turn is repeated here on the next send */
  ]
}
```

4. Current response - this is small

Success

```
{ "text": "Claro. Fumador o no fumador?" }
```

Failure (timeout, empty key, non-200, no candidates) - same as Dart: null

```
{ "text": null }
```

5. How load grows (same API, bigger body)

User send #	What Flutter puts on POST /v1/chat	What Gemini generateContent receives	What the API returns
1	prompt + 0 history + new message	prompt + 1 user turn	one new text (or null)
5	prompt + 8 old turns + new message	prompt + those 8 + new user turn	one new text (or null)
20	prompt + ~38 old turns + new message	the same large contents array	one new text (or null)

The Node server also parses a larger JSON each time. That is still cheap compared with Gemini input tokens. The serious load is: mobile upload of history + Gemini input size. Not the response JSON.

6. Proposed smaller request (Firestore already stores the chat)

Do not change Gemini mapping (full history still goes to generateContent unless a later, separate decision says otherwise). Only stop sending that history from the phone on every POST.

Once: POST /v1/conversations

```
{ "mode": "role_play", "systemInstruction": "<RolePlayGeminiPrompt.build()>", "ownerId": "uid" }
201 { "conversationId": "chat_123" }
```

Every later message: POST /v1/conversations/chat_123/messages

```
Request: { "userMessage": "Una mesa para dos" }
Backend: Firestore READ prompt + full ordered messages, then generateContent, then WRITE if
success.
Response: { "text": "..." } or { "text": null }
```

7. Unchanged Gemini rules (from submitted Dart)

History turns first, current user last; roles user / model; four safety categories BLOCK_MEDIUM_AND_ABOVE; first candidate, first text part, trimmed. Role-play and free-chat English prompts unchanged. This file is not a Groq, streaming, dashboard, or chat-summary spec.

Source: GeminiService.generateReply, RolePlayGeminiPrompt, FreeChatGeminiPrompt.